Gen 3 Hayabusa Race Patch

Speed density • rolling anti-lag • air-shifter • boost control

This patch set turns a stock Gen 3 Hayabusa ECU read (5JCZ…) into a boosted race calibration in one command. It swaps the fuelling to speed density on a 3-bar MAP sensor, disregards the stock throttle (alpha-N / TPS) fuel path, and adds three code features: rolling anti-lag, an auto-upshift air-shifter, and gear-scheduled boost control on a repurposed output. All are applied by a script that re-stamps the ECU checksum for you.

EVERYTHING HERE IS UNVERIFIED ON HARDWARE. Bench a spare ECM first, keep your stock read for recovery, and confirm every output on the bench before the engine runs. A boosted, speed-density, uncalibrated tune can hurt the engine - this is a starting point for a tuner, not a finished map.

Files you need

File	What it is
tools/apply_patches.py	Applies the patches to a stock bin and re-stamps the field-1 CRC.
tools/fix_field1_crc.py	Re-stamps the CRC after you edit a bin (TunerPro does not).
race/Busa-SD-v6.xdf	TunerPro definition - open your bin with this to enable & tune.
race/Busa-Race-v1.bin	Example fully-configured race bin (all four features set up).
race/notes/*.txt	Byte-level notes per feature (boost.txt, autoshift.txt, race-v1.txt, SD-v*.txt).

1 Apply the patch

From the `hayabusa/` folder, build a race bin from any stock read:

```
python3 tools/apply_patches.py stock.bin race.bin
  # all four: speed-density + anti-lag + air-shift + boost

python3 tools/apply_patches.py stock.bin out.bin sd antilag
  # pick patches: any of sd antilag autoshift boost

python3 tools/apply_patches.py stock.bin out.bin sd --sensor gm25bar
  # speed density on a GM 2.5-bar sensor (default is gm3bar)

python3 tools/apply_patches.py race.bin x --check
  # report what is already applied, write nothing
```

- The script checks the original bytes before writing and **aborts safely** if the read is not supported (nothing is written).
- The **anti-lag**, **air-shift** and **boost** features are code in free flash and drop onto **any** of the eight reads. **Speed density** replaces fuelling calibration taken from 5JCZSJ40; that calibration is shared by every read **except 5JCZSJB0**.
- After any edit inside TunerPro, re-stamp the checksum: `python3 tools/fix_field1_crc.py your.bin` – or the ECU may reject the bin.
- All three engine features ship **disabled**; you turn them on in the XDF after bench checks.

Flash order for a first bring-up: a known-good CKTEST / bench bin first (proves the flash path and that the hardware checksum is not enforced), then your patched bin with everything still disabled, then enable one feature at a time.

2 Speed density & the MAP sensor

The **sd** patch rescales the manifold-pressure (IAP) input for a wide-range MAP sensor, repoints the fuel lookups from manifold vacuum to absolute pressure, and **disregards the throttle (TPS / alpha-N) fuel path** so the MAP fuel tables have authority everywhere - idle to full boost. It also adds boost spark-retard maps. Fuel values are a conservative starting point and **need a wideband**; injector scaling is still stock, so set your injector size and dead-times before adding boost.

Pick / calibrate the MAP sensor

Choose the sensor with `--sensor`. Only the 16-bit gain/offset differ; the preset guide and formula live in the XDF item *IAP Sensor :: Conversion Divisor*.

--sensor	Transfer kPa = a*raw + b	GAIN	OFFSET
gm3bar (default)	a=0.307429, b=-12.6 (~20-300 kPa)	0xA12F	0x6CA6
gm25bar	a=0.256598, b=-10.5 (~20-250 kPa)	0x8689	0x6FE0

Any sensor: `GAIN = round(393.14*a*10925/32)`, `OFFSET = round(32768 + 393.14*b)`.

Both presets are PROVISIONAL. Before trusting any fuel number, key-on and confirm raw*a + b equals local barometric pressure; adjust a, b and recompute if not, and match the kPa equation on the SD fuel / boost maps.

3 Enable & tune in TunerPro

Open your patched bin with `race/Busa-SD-v6.xdf`. The feature settings are in the folder **"22 Patches & Air-Shifter"**. Turn each flag to 0x80 to enable. Re-stamp the CRC after saving.

Rolling anti-lag (`ALS …`)

Rolling at or above the set speed, hold START and go WOT: the ECU captures that RPM and holds it with an all-cylinder spark cut (fuel stays on). Release START for full power.

Spark-cut anti-lag with fuel on dumps raw fuel into a hot exhaust - big EGT and turbo / exhaust stress. Use short bursts; the max-active timer is a backstop, not a duty cycle.

Air-shifter / auto-upshift (`Auto-Shift :: …`)

At each per-gear RPM target it pulses the PAIR output to drive a relay -> MAC valve -> air ram on the shift lever. No spark cut of its own - it relies on the factory quickshifter. Confirm the output pin with *Auto-Shift :: Bench Test Output* before trusting it.

Boost control (`Boost :: …`)

Software-PWMs a race-unused output to a wastegate / boost-control solenoid, with the duty scheduled **per gear** so you can stage boost as traction allows. An optional closed loop trims toward a target read from a pressure

sensor (see section 4). Key settings:

Boost :: item	Meaning
Enable	0x80 = on. Starts the per-gear PWM (output still must be bench-confirmed).
Bench Test Output	0x80 = hold solenoid asserted to find its pin. Polarity is set by wiring, not here.
PWM Period	Software-PWM period in ~5 ms ticks (20 ~ 10 Hz). Duty is 0..Period.
Min RPM to Run	Below this the solenoid is off (wastegate spring pressure = least boost).
Duty Gear 1..6	Open-loop feed-forward on-ticks per period in each gear. duty% = ticks / Period.
Closed-Loop Enable	0x80 = trim duty toward the Boost Target table. Leave off until the inputs are proven.
Boost Target :: Mode A/B/C	Three gear x RPM target maps (one per power mode), entered in kPa. The closed loop trims toward the cell for
CL Trim Step	Duty ticks added / removed each period while correcting (clamped +/- Period).
Pressure Input Address	Which ADC word the pressure sensor is read from (see section 4).
Mode Input Address	Which RAM byte holds the active power mode (PW 1-3 -> table Mode A/B/C). Default 0xFEF01F74.

Which way duty moves boost depends on your solenoid plumbing. On a normally-closed MAC valve bleeding the wastegate signal, more duty = more boost; confirm yours and flip the per-gear numbers if it is the other way. Keep a wastegate spring that is safe at 0 % duty - the solenoid controls boost, it does not protect the engine.

4 Wastegate / boost pressure sensor

On a turbo build the stock narrowband O2 sensors are unusable and are normally deleted, which frees both halves of an O2 circuit. The boost patch repurposes them: the O2 **heater driver** becomes the solenoid output, and the O2 **signal input** becomes a 0-5 V boost-pressure input for closed loop. The pressure sensor is fully configurable without re-assembling code:

- **Input channel** – set *Boost :: Pressure Input Address* to the low 16 bits of the ADC RAM word the bench shows your sensor on (effective address = 0xFEF00000 + that value). Default 0x2648 -> 0xFEF02648 is a candidate; confirm it by feeding the input a known voltage and watching the value.
- **Scaling** – the *Boost Target* cells store raw ADC counts but their equation is your sensor's transfer, so you type targets in kPa. Set that equation to match your sensor; GM-style starting points are 3-bar ~ X*0.342-15 and 2.5-bar ~ X*0.285-12. Calibrate against a known pressure.
- **Power mode** – *Boost :: Mode Input Address* (default 0xFEF01F74) is the RAM byte holding the active power mode (PW 1-3); it picks the Boost Target table Mode A/B/C, so you can run more boost in A than C.
- **Live readout** – add a RAM gauge at 0xFEF0<your address> with the same equation to watch boost while you tune.

Leave Closed-Loop Enable OFF until the input is proven on the bench. Open-loop per-gear duty works without any pressure sensor and is the recommended first step.

5 Bench bring-up checklist

Step	Do this	Pass when
1	Flash a CKTEST / bench bin.	It boots - flash path + checksums OK.

Step	Do this	Pass when
2	Flash your patched bin, everything disabled.	Boots; key-on raw IAP*a + b = baro.
3	Idle on the stand.	Runs; log MAP, fuel source, AFR.
4	Air-shift: set Bench Test Output = 0x80.	The PAIR pin / relay clicks, nothing else moves.
5	Boost: set Boost Bench Test Output = 0x80.	The chosen O2-heater pin drives the solenoid.
6	Boost pressure input: feed a known voltage.	The chosen ADC word tracks it.
7	Enable one feature at a time, on a wideband.	Behaves as set; no knock, AFR in range.

6 Example configuration (Busa-Race-v1.bin)

The repo ships one fully-configured example. It is a plausible working setup, not a finished tune - calibrate injectors + sensor and bench the outputs first.

Feature	Set to
Speed density	GM 3-bar MAP; throttle fuel path off; SD starter fuel maps.
Anti-lag	Armed: min 20 km/h, capture 90 deg WOT, 3000-9500 rpm, 10 s max.
Air-shifter	Armed: ~100 ms pulse, 30 deg WOT gate, shift at 10,500 rpm all gears.
Boost (open loop)	Armed: 10 Hz PWM, off below 3500 rpm; duty 20/35/50/65/80/90 % in gears 1-6.
Boost (closed loop)	Off; target tables populated per mode (A full / B mid / C low), inputs 0xFEF02648 + 0xFEF01F74 - enable after pr

Rebuild it yourself:

```
python3 tools/apply_patches.py stock/5JCZSJ40/5JCZSJ40.bin race/Busa-Race-v1.bin
# then set the values above in the XDF, then fix_field1_crc.py
```

Full byte-level detail per feature is in race/notes/ (boost.txt, autoshift.txt, SD-v1..v4.1.txt, race-v1.txt). Source for the code patches is in race/als/ (als.s, autoshift.s, boost.s; build with build.sh on a v850e3v5 toolchain).